Functional Verification of Smart Contracts via Strong Data Integrity

Wolfgang Ahrendt[1(✉)] and Richard Bubel[2(✉)]

[1] Chalmers Technical University, Gothenburg, Sweden
ahrendt@chalmers.se
[2] Technical University Darmstadt, Darmstadt, Germany
bubel@cs.tu-darmstadt.de

Abstract. We present an invariant-based specification and verification methodology that allows us to conveniently specify and verify strong data integrity properties for Solidity smart contracts. Our approach is able to reason precisely about arbitrary usage of the contracts, which may include re-entrance, a common security pitfall in smart contracts. We implemented the approach in a prototype verification tool, called SolidiKeY, and applied it successfully to a number of smart contracts.

1 Introduction

Distributed ledger technology (DLT) has been around since the incarnation of Bitcoin by Satoshi Nakamoto. The crucial idea was to publicly agree on, and irreversibly record, the content and order of transactions. The backbone of DLT is an immutable list data-structure called blockchain. The concept of *smart contracts* was coined by Nick Szabo prior to the rise of DLT. Their DLT incarnation, however, is based on the realisation that the blockchain concept can also store programs, as well as their runtime state in between transactional units of execution. These programs are called (smart) contracts because they are meant to form an agreement of procedure between all parties which choose to engage with them. The first, and still major, DLT framework which supports smart contracts was Ethereum [16], with its built-in cryptocurrency Ether. The virtual machine on which smart contracts are run is called Ethereum Virtual Machine (EVM). In Ethereum not only the users, but also the contracts themselves can receive, own, and send Ether. Sending Ether to a contract, and calling the contract, is the same thing in this framework. Sending funds to a contract without passing control to the receiver is not possible.

Ethereum miners look for transaction requests on the network. A transaction request contains the address of a contract to be called, the call data, and the amount of Ether to be sent. Miners are paid for their efforts with units of "gas", to be paid by the address which requested the transaction. Each EVM instruction has an assigned cost. If the execution runs out of gas, the currently executing transaction is aborted, discarded, and the state is reverted to before the start of the transaction (except for the depleted gas). Lack of gas is only one of numerous

T. Margaria and B. Steffen (Eds.): ISoLA 2020, LNCS 12478, pp. 9–24, 2020.
https://doi.org/10.1007/978-3-030-61467-6_2

causes for aborting and reverting a transaction. Others are attempts to send unbacked funds, or failing runtime checks.

Even if contracts can be written directly in EVM bytecode [16], there are several higher level languages that are compiled into EVM bytecode. The most popular one is Solidity [6], which has been inspired by C++, Python, and JavaScript.

One important difference between traditional programs and Ethereum smart contracts is that the latter do not only pass around information, but assets, in particular crypto funds. Many interesting, contract specific properties, on which users want to rely, depend on the flow of assets, and the interaction of the asset flow with the internal state of the contracts. In this work, we put the spotlight on the *relation between the flow of Ether and the data of the contract.*

We report on a methodology, logic, and calculus for Solidity source code verification. Invariants describe the expected relation between the Ether flow and the contract's data. These invariants should hold independent of usage scenarios (intended or malicious) and cover every behaviour. Our approach guarantees *strong data-integrity* properties, and supports full functional verification. The calculus keeps track of ingoing and outgoing payments, in order to prove that the payment-data-invariant is intact at critical points. The precise points, where such invariants have to hold, and where they can be assumed again, must fully reflect the subtleties of Solidity and the EVM. We must ensure that data integrity is independent of the (potentially hostile) environment.

A central feature, for specification and verification, will be easy access to the *financial net* (difference between incoming and outgoing funds) which the examined contract has with every address in its environment. We specify invariants and postconditions with help of the financial net, and verify them in a calculus which knows when and how to manipulate the net.

The methodology has been implemented in a prototype of what is going to become a fully fledged verification tool for smart contracts written in Solidity, called SolidiKeY. It is based on the state-of-the-art deductive verification system KeY [1]. We also report on the successful application of SolidiKeY on example contracts. Although the calculus does not yet cover Solidity fully, the methodology, and most aspects of the calculus, will remain valid for future developments of the approach and tool.

2 Background

2.1 Solidity

We cannot afford a proper introduction to Solidity. Many things become clear when discussing examples (Figs. 1 and 4). Here, we only mention a few concepts.

Solidity follows largely an object-oriented paradigm. The equivalent to object fields is called state variables, but we call them fields throughout this paper. External users and contract instances—think of the latter as objects—share the same type, `address`. (More precisely, contracts are convertible to `address`.) Each `address` owns Ether (possibly 0), can send Ether to other addresses, and receive Ether. The type `address payable` is like `address`, but with the additional

members `transfer` and `send`, used for receiving Ether. For instance, a`.transfer(`v`)` transfers the amount of v Ether to a. Contracts can be called by every address (incl. contracts), receiving Ether along with the call. What is called method in object-orientation is called function in Solidity. Built-in data types include unsigned integer (`uint`), dynamic and fixed-size arrays, enums, structs, and mappings. Mappings associate keys with values. For instance, the declaration `mapping (address => uint) public balances` declares a field `balances` which contains a mapping from addresses to unsigned integers. Storing and lookup in mappings uses array-like syntax. The modifier `public` does *not* mean that the field is writable by anyone. Instead, it means that a public getter function for `balances[x]` is automatically provided. The current caller, and the amount of Ether sent with the call, are always available via `msg.sender` and `msg.value`, respectively. Only `payable` functions accept payments. `require(`b`)` and `assert(`b`)` check the boolean expression b, and abort (and revert) when b is false. Apart from how much gas is kept, behaviour is the same. However, what differs is the implication of a failure to program correctness. A failing `require(`b`)` is considered the caller's problem ("If you call me like that, there is nothing I can do for you."). In contrast, `assert(`b`)` is a claim of the contract's implementer that b will always be true at this point. Breaking such an assertion indicates a bug in the implementation. Solidity features programmable *modifiers*, offering some kind of delta-oriented programming. This will be explained in Sect. 5. Finally, in Solidity terminology, *storage* refers to the data stored in the fields of a contract, whereas *memory* refers to local variables and their values.

2.2 Dynamic Logic

To verify that programs adhere to their specification, we use *dynamic logic* (DL), a program logic which, like Hoare logic, expresses properties of programs, and the programs themselves, inside the same formula. We follow the tradition of the KeY approach and system [1], which targets object-oriented programs. KeY is a particular good fit as Solidity's contract-oriented approach shares many principles with object-orientation.

The calculus and the prototype of the verification system SolidiKeY do not yet cover full Solidity. Further features will be covered in future work. For now, we refer to our calculus as a 'calculus for Solidity Light' (see Sect. 4). Solidity Light features a simpler memory model (no distinction of storage and memory, reference semantics for all assignments to non-primitive types, like in Java), and unbounded integers. This allows us to focus, for now, on the main contribution of this paper, namely a general specification and verification methodology for smart contracts to ensure strong data integrity. The simpler memory model does not impact provability as long as we refrain from assignments to non-primitive typed fields as a whole. Solidity's bounded (round-robin) integers can be easily added in the future, following the lines of KeY's existing support for Java's round-robin integers. Both limitations will be addressed in future work.

Syntax. Dynamic logic (DL) for Solidity is an extension of first-order logic (FOL) by the box *modality* and by *updates*. Every FOL formula is a DL formula. The *box* modality $[\cdot_1]\cdot_2$ takes as first argument (a fragment of) a Solidity program, and as second argument another DL formula. The formula $[s]\mathsf{post}$ means: *if* s terminates successfully (i.e., s does not revert), then the property post holds in the reached final state. An important special case of DL formulas is the pattern $\mathsf{pre} \rightarrow [s]\mathsf{post}$, which in Hoare triple notation would be $\{\mathsf{pre}\}s\{\mathsf{post}\}$.

In order to enable symbolic execution as a proof principle, KeY style DLs are further extended with *updates*. An update $v_1 := t_1 \| \ldots \| v_n := t_n$ consists of a program variable $v_1, \ldots, v_n$ and a terms $t_1, \ldots, t_n$. (n can be 1.) Updates represent state changes and are essentially explicit substitutions. An update u can be applied to a formula ϕ, written $\{u\}\phi$, resulting again in a formula. Evaluating $\{v_1 := t_1 \| \ldots \| v_n := t_n\}\phi$ in a state s is equivalent to evaluating ϕ in a state s', where s' coincides with s except that each v_i is mapped to the value of t_i. (Different occurrences of identical v_i, v_j are resolved by "right-win".)

Example 1. Let `x`,`y`, and `z` be program variables. The meaning of the DL formula $\mathtt{x} \leq \mathtt{y} \rightarrow [\mathtt{t=x;x=y;y=z;}]\mathtt{y} \leq \mathtt{x}$ is that if the three assignments are executed in a state where $\mathtt{x} \leq \mathtt{y}$, then $\mathtt{y} \leq \mathtt{x}$ holds after the execution. Turning the first assignment into an update, we get the formula $\mathtt{x} \leq \mathtt{y} \rightarrow \{\mathtt{t} := \mathtt{x}\}[\mathtt{x=y;y=z;}]\mathtt{y} \leq \mathtt{x}$. After processing the other two assignments in the same way, we get the formula $\mathtt{x} \leq \mathtt{y} \rightarrow \{\mathtt{t} := \mathtt{x}\}\{\mathtt{x} := \mathtt{y}\}\{\mathtt{y} := \mathtt{z}\}\mathtt{y} \leq \mathtt{x}$. Applying the updates to each other, composing the substitutions, we get $\mathtt{x} \leq \mathtt{y} \rightarrow \{\mathtt{t} := \mathtt{x} \| \mathtt{x} := \mathtt{y} \| \mathtt{y} := \mathtt{x}\}\mathtt{y} \leq \mathtt{x}$. Applying the (now single) update, as a substitution, to the postcondition, results in $\mathtt{x} \leq \mathtt{y} \rightarrow \mathtt{x} \leq \mathtt{y}$, which is trivially true.

Calculus. To reason about the validity of formulas, we use a *sequent calculus*. A *sequent* $\phi_1, \ldots, \phi_n \Longrightarrow \psi_1, \ldots, \psi_m$ consists of two sets of formulas called antecedent ($\phi_1, \ldots, \phi_n$) and succedent ($\psi_1, \ldots, \psi_m$). Its meaning is equivalent to the formula $\bigwedge_{i=1..n} \phi_i \rightarrow \bigvee_{i=1..m} \psi_i$. Calculus rules are denoted as rule schemata

$$\mathsf{name}\ \frac{\overbrace{\Gamma_1 \Longrightarrow \Delta_1 \quad \ldots \quad \Gamma_n \Longrightarrow \Delta_n}^{premises}}{\underbrace{\Gamma \Longrightarrow \Delta}_{conclusion}}$$

where $\Gamma, \Delta, \Gamma_i, \Delta_i$ with $i = 1..n$ are schemavariables denoting sets of formulas.

A proof is a tree whose nodes are labelled with sequents. For each inner node there is a rule r such that its conclusion matches the node's sequent and the node's children are labelled with the instantiated premises of r. Applying a rule with no premises closes the branch. A proof is closed if all branches are closed. Our calculus design follows the symbolic execution paradigm. The rules realise a symbolic interpreter by step-wise rewriting the first statement of a program into a sequence of simpler statements until an *atomic* statement is reached, which is then translated into logic possibly using updates and proof branching.

We illustrate this approach with the following two sequent rules:

ifThenElse
$$\frac{\Gamma, \{u\}(b = \text{true}) \Longrightarrow \{u\}[s1\,;\,\omega]\phi, \Delta \quad \Gamma, \{u\}(b = \text{false}) \Longrightarrow \{u\}[s2\,;\,\omega]\phi, \Delta}{\Gamma \Longrightarrow \{u\}[\texttt{if (}b\texttt{) \{ }s1\texttt{; \} else \{ }s2\texttt{; \} }\omega]\phi, \Delta}$$

assign
$$\frac{\Gamma \Longrightarrow \{u\}\{v := se\}[\omega]\phi, \Delta}{\Gamma \Longrightarrow \{u\}[v = se\,;\,\omega]\phi, \Delta}$$

Schemavariable ω matches the remainder of the program, schemavariable u matches the leading update, b, v match program variables, and se matches on simple expressions without side-effects. In the above rules, u represents the condensed effect of symbolically executing the program up to now, and the next statement to be symbolically executed is the if-statement, or assignment, respectively.

Storage Modelling. Solidity Light distinguishes between storage, which consists of the fields and their values, and local variables. Unlike Solidity, it uses reference semantics for all assignments. This does not affect the paper's main contribution for which the storage details are orthogonal. We model the storage as data structure of type `Storage` axiomatised as an algebra of arrays. If a program accesses a field, symbolic execution reads from, or writes to, the data structure referred to by the global program variable `storage`.

3 Verifying Strong Data Integrity of Smart Contracts

Consider the Solidity contract `Auction` given in Fig. 1, realising an auction dealing with, and remembering, multiple bidders and their bids. The implementation choices (including the violation of some best practices) serve the upcoming discussion. The contract can be used by anyone to place or increase a bid with the same function. Bidders can also withdraw their (accumulated) bid at any time. The auction owner can close the auction, in which case the highest bid is sent to the owner, and all bidders, except the highest bidder, are reimbursed.

The implementation of `Auction` is centred around the field `balances`, a mapping from addresses to (unsigned) integer. `balances` *is meant to store the accumulated funds sent by each bidder, minus anything that has been sent back.* This property, let us call it I, is what the implementer aimed to keep intact, by increasing or decreasing `balances[`x`]` whenever funds flow from or to x, respectively. Also, the users' expectations on the contract are based on I. For instance, the bidder's expectation to get back all the investments when `withdraw`ing (see line 27) is based on the understanding that I is kept intact throughout the lifetime of the contract. The same holds for the expectation of the auction owner to receive the investments of the winner, and all (non-winning) bidders' expectation of getting back their investments in `closeAuction` (see lines 35, 39).

It is the fulfilment of such expectations which we call *functional correctness* of a contract. In turn, functional correctness depends on keeping properties, such as I in our example, intact. Such properties form a relation between internal

```
contract Auction {
 address payable private auctionOwner;
 mapping (address => uint) public balances;
 mapping (address => bool) private bidded;

 uint numberOfBidders;
 address payable [100] public bidders;

 constructor() public
  { auctionOwner = msg.sender; }

 function placeOrIncreaseBid()
  public payable {
   balances[msg.sender] =
    balances[msg.sender] + msg.value;
   // If caller didn't bid yet,
   // add her to bidders
   if (!bidded[msg.sender]) {
    bidders[numberOfBidders] = msg.sender;
    numberOfBidders = numberOfBidders + 1;
    bidded[msg.sender] = true;
   }
 }
 function withdraw() public {
  // A bidder can withdraw all her money
  require(bidded[msg.sender]);
  msg.sender.transfer(balances[msg.sender]);
  balances[msg.sender] = 0;
 }

 function closeAuction() public {
  // Determine highestBid and winner
  ...
  // Transfer the money to the auction owner
  auctionOwner.transfer(highestBid);
  balances[winner] = 0;
  // Reimburse everyone else
  for(i = 0; i < numberOfBidders; i = i + 1) {
   bidders[i].transfer(balances[bidders[i]]);
   balances[bidders[i]] = 0;
  }
 }
}
```

Fig. 1. Contract auction

data (fields) and the history of payments. This paper is mostly concerned with proving that these relations between data and history are invariants. This means that they are established at program points where control is potentially passed to another contract (and reentrance may occur). This allows us to safely assume the validity of these relations once control is regained. In other words, we prove that no usage of the contract can break these relations no matter if by a simple programming error or an attack. The invariance of such properties is what we call *strong data integrity*. And we call the properties themselves *invariants*.

Let us look closer at the informal invariant I: "`balances` *stores for each bidder the accumulated funds sent by the bidder, minus anything that has been sent back.*" As a step towards formalising this invariant, we could express I as:

$$\forall a \in \text{address}.\quad \texttt{balances}[a] = \sum\{\texttt{msg.value} \mid \texttt{msg.sender} = a\} - \sum\{v \mid a.\texttt{transfer}(v)\} \quad (1)$$

Note that we are not really using a formal language here, just mathematical notation to convey the meaning of I a bit more precise than in natural language. Formula (1) says that, for all addresses a, the value stored in the map `balances`, under key a, corresponds to the difference of two sums. The first sum adds up all payments sent by a up to now, in the history of this contract's existence. The second sum adds up all values that have been sent to a, via `transfer`, up to now. The difference between the two is what is supposed to be stored in $\texttt{balances}[a]$. This holds even for addresses which did not bid (yet), as maps with integer value type are implicitly initialised to value 0. However, (1) does not actually hold for the winner or auction owner after closing the auction. We will come back to that.

As we said before, the functional correctness of the contract relies on I being an invariant of the contract. To verify this, with a verification tool, we need a

language to specify such properties, and to reason about them in a suitable logic and calculus. One could base such a language on concepts which are present in (1), however being more explicit and precise. Such a language would have as a datatype the history of call (i.e., payment) events. Also, it would allow to project out specific call events, like in the set comprehension in (1), select values of interest, and feed them to a computation, like the summations in (1). This can be done, and has been done in the compositional verification of functional properties of distributed programs (see, e.g., [3,7]). Unfortunately, the experience with compositional deductive verification approaches where the call history is an explicit piece of data shows that specifications tend to get complex, and verification meets challenges concerning efficiency and usability.

While information about past (in- and out-going) calls is essential, we believe that the full power of the call/payment history will typically not be necessary to formulate and verify strong data integrity, and with that, the implied functional properties. It can suffice to reason about the financial *net*, i.e., the difference between in- and out-going payments, which a contract has with any other address. For that, our logic uses the built-in mapping $\texttt{net} : \text{address} \to \mathbb{Z}$ to keep track of the money flow between this contract and other addresses, with

$$\mathsf{net}(a) = \text{money received from } a - \text{money sent to } a$$

With the help of the net mapping, property (1) becomes simply:

$$\forall a \in \text{address}.\ \texttt{balances}[a] = \mathsf{net}(a) \tag{2}$$

Now that we can express formulas more concisely, let us get the data integrity property of Auction more precise. As indicated earlier, the above invariant does not always hold for the highest bidder hb, neither for the auction owner. We refine the invariant (2) accordingly, exploiting again the power of the net mapping:

$$\begin{aligned} &\exists hb \in \text{address}.\ \forall a \in \text{address}.\ \texttt{balances}[hb] \geq \texttt{balances}[a] \\ &\quad \wedge\ (\ a \neq hb \wedge a \neq \texttt{auctionOwner} \to \texttt{balances}[a] = \mathsf{net}(a)\ ) \\ &\quad \wedge\ \texttt{balances}[hb] = \mathsf{net}(hb) + \mathsf{net}(\texttt{auctionOwner}) \end{aligned} \tag{3}$$

The last line captures different situations at once. All the way until closing the auction, $\mathsf{net}(\texttt{auctionOwner})$ is 0, and $\texttt{balances}[hb] = \mathsf{net}(hb)$, just like for other bidders. After closing the auction, $\mathsf{net}(\texttt{auctionOwner})$ is $-\mathsf{net}(hb)$, as the investment of the highest bidder were sent to the owner, and $\texttt{balances}[hb]$ is 0.

For a given contract, once we capture the specific notion of data integrity in a formula I, how can we verify that I indeed *is* an invariant of the contract? Where does an invariant need to hold? Obviously, such invariants do not need to hold at each point in time. For instance, consider a situation where placeOrIncreaseBid() has just been called, the payment has been received, and the program counter is at line 15 (Fig. 1), i.e., the assignment has not been executed yet. At this point, the invariant is not intact. The relation between payments and the contract data is out of sync, but that is all right, because it is repaired (by executing line 15) before control is given away again.

A verification methodology needs to make sure that the invariant of contract C holds whenever control is *outside* C, like after C has been created, and before and after each transition executing C, but also during a transition while some other contract executes. We visualise the invariant verification discipline in Fig. 2. It shows schematically the code of a function `f` (of the contract C we want to verify). `f` contains two calls to other contracts, `c1.f1()` and `c2.f2()`. We assume the invariant I to hold *before* `f` is called and funds are passed to C. After the funds were passed, I may be broken, which is fine as long as we show that I is intact again when `c1` gains control, *after* funds were passed to `c1`. (If instead we showed I before funds were passed, I would not be intact once `c1` gains control.) We assume I when `c1.f1()` returns, and have to show I once funds were passed to `c2`, and assume it again when `c2.f2()` returns. Finally, we have to show that I holds at the end of `f`. If we follow such a principle for all `public/external` functions of C (and in addition show that the `constructor` establishes I), we have proven that I is intact whenever control is outside C, in particular after each transaction on C. Note that this reasoning also covers the case where, in the above example, `c1.f1()` calls back into any function of C. In fact, the proof principle does not rely on any particular ordering of code snippets (between gaining and releasing control). Hence, verified invariants, and implied functional properties, cannot be compromised by re-entrancy.

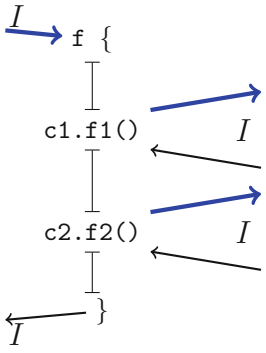

Fig. 2. Assuming and proving invariant I

It is important to note that a.`transfer`(`v`) is a *call* (of a special kind) to a. Therefore, invariants also need to be shown when executing a.`transfer`(`v`) (but after v has been passed, see above). We discuss a possible weakening later on.

So can we verify the invariance of property (3) in `Auction` with this discipline? The answer is no. Let us assume (3) holds when `withdraw` is called (line 24). Thereby, `balances`[`msg.sender`] = `net`(`msg.sender`) holds before executing line 27. But when executing line 27, in between sending money to `msg.sender` and the return of `transfer`, the `net`(`msg.sender`) is 0, whereas `balances`[`msg.sender`] is unchanged, which breaks the invariant (3). If for instance `msg.sender`, before returning from `transfer`, calls back into `Auction`, the assumption, intuitive or formal, that contract data and payments are in sync at the beginning of each function has become a harmful illusion. `Auction` does not feature strong data integrity! This can be repaired, by updating `balances`[a] *before* sending funds to a. With that, (3) can be shown correct after the passing of funds, and our verification discipline guarantees data integrity for `Auction`.

Note that only successful, non-reverting code execution needs to maintain the invariant of a contract. The reason is that reverting executions (with for instance unbacked payments, or failing `require`s) are discarded, and the state is reset to before the failing transaction, where we can assume the invariant again.

4 A Calculus for Solidity Light

4.1 From Specification to Proof-Obligation

To verify that a contract satisfies its specification, we construct, for each function, a dynamic logic formula (called *proof obligation*) from the function and the specification. If all functions (plus the constructor) of a contract have been verified, then the contract as a whole adheres to its specification. In the current SolidiKeY prototype, however, specifications are manually translated to proof obligations in Solidity dynamic logic. (In a mature verification system, specifications will be automatically translated to proof obligations.)

```
1  contract C {
2    //@ invariant I;
3
4    T field_1;
5    ...
6    T field_n;
7
8    /*@ requires
9      @   pre
10     @ after_success
11     @   post
12     @*/
13   function f(T args)
14             modifiers { body }
15 }
```

Fig. 3. Contract w/ specification

In order to explain how the proof obligation is constructed, consider a contract C, annotated with specification, as shown in Fig. 3. The annotated contract specifies a *contract invariant* I, as well as *function specifications* for functions, here f. The function specification states that if f is called in a state satisfying precondition *pre*, and in which I holds (for the storage and payment history of `this` contract object), then the execution of f, if successful, guarantees that postcondition *post* and invariant I are satisfied in the post-state.

The contract's semantics can now be expressed by instantiating the proof obligation schemata (4) with the concrete specification:

$$\begin{aligned} &\texttt{msg.value} \geq 0 \ \wedge \ I \ \wedge \ \mathit{pre} \ \rightarrow \\ &\{\texttt{old} := \texttt{storage}\}\{\texttt{net}(\texttt{msg.sender}) := \texttt{net}(\texttt{msg.sender}) + \texttt{msg.value})\} \\ &\qquad [\texttt{result = } f(\texttt{msg}, \overline{\mathit{args}})@C;](I \wedge \mathit{post}) \end{aligned} \tag{4}$$

The variable `old` saves the storage before the function is called and can be used in the postcondition to refer to values of fields (incl. arrays, structs, and mappings) in the pre-state. This allows us for instance to express that the value of a field got increased by some value. Each function has an additional argument `msg` used to pass the address of the caller and the amount of passed Ether. The update on `storage` reflects that the net of the callee changes due to the payment which comes with the call. As explained in Sect. 3 the invariant is assumed before the transfer of the Ether. In case of a non-payable function f, the proof obligation is strengthened by changing $\texttt{msg.value} \geq 0$ to $\texttt{msg.value} = 0$.

4.2 Symbolic Execution Rules for Contract Execution

To prove the formula valid, we use a sequent calculus as outlined in Sect. 2.2. Here, we focus on the symbolic execution rules for Solidity, highlighting where the specific pitfalls lie when the underlying virtual machine is blockchain based. Symbolic execution of a function consists of:

1. left-to-right evaluation of the function arguments,
2. symbolic execution of the function implementation.

As for built-in "functions" (e.g., `assert` and `require`), their meaning is reflected by providing specific rules in the calculus.

Left-to-right evaluation of the function call arguments is realised by rule unfoldArgument. The rule rewrites a function call with complex arguments into a sequence of statements. For each argument a new local variable of compatible type is introduced and initialised with the corresponding argument expression. The assignment order ensures the correct evaluation order of the arguments. The arguments in the method call are replaced by the fresh variables:

$$\text{unfoldArgument}\ \frac{\Gamma \Longrightarrow \{u\}[T\ \ x;\ \ x\ \texttt{=}\ e;\ \ m(x);\ \ \omega]\phi, \Delta}{\Gamma \Longrightarrow \{u\}[m(e);\ \ \omega]\phi, \Delta}\quad e \text{ complex}$$

Built-in functions `require` *and* `assert`. Both built-in functions `require` and `assert` check whether the Boolean expression given as argument evaluates to true, and abort if otherwise, with the state being reverted.

Although their operational semantics is identical (except for gas handling), their pragmatics differ in the sense that `require` is used as a kind of refusal to execute a certain call unless the caller made sure that a certain condition holds. Failing `require`s are not considered harmful, and nothing is wrong with the inspected code. (If there is any blame, than it is on the callers side.) In contrast, `assert` is used as a claim that a certain property always holds at a specific code position. Failing `assert`s manifest an error in the inspected code base, and our calculus treats it accordingly. Here comes the rule for `require`:

$$\text{require}\ \frac{\Gamma, \{u\}\texttt{b} = \text{true} \Longrightarrow [\omega]\phi, \Delta}{\Gamma \Longrightarrow \{u\}[\texttt{require(b);}\ \ \omega]\phi, \Delta}\quad \texttt{b} \text{ simple}$$

Turning `b` into an assumption is sound as we are only examining successful transactions (partial correctness), and thus a reverted transaction is trivially correct. In particular, a reverted transaction maintains every invariant, because it is undone and has no effect (other than gas consumption, which we do not talk about in the invariants). This observation is important for the overall invariant verification discipline, and carries over to other statements which may possibly lead to reverting a transaction, like the attempt to send unbacked funds with `transfer`, see below.

In contrast the rule for `assert`

$$\text{assert}\ \frac{\Gamma \Longrightarrow \{u\}\texttt{b} = \text{true}, \Delta \qquad \Gamma, \{u\}\texttt{b} = \text{true} \Longrightarrow [\omega]\phi, \Delta}{\Gamma \Longrightarrow \{u\}[\texttt{assert(b);}\ \ \omega]\phi, \Delta}\quad \texttt{b} \text{ simple}$$

splits the proof, and demands proving that `b` is true in the current symbolic state, while continuing symbolic execution on the second branch under the (then justified) assumption that `b` is true. This semantics prevents us from closing a proof if an assertion does not hold, thereby detecting further programming errors.

Function Calls Transferring Ether. Transfer of resources, for instance cryptocurrencies (here: Ether), is central to most distributed ledger frameworks. In Solidity, Ether is transferred to a contract by calling one of its `payable` functions. Any function declared in a contract can be marked `payable` by using the eponymous modifier. Calling these functions is usually done using the `call` statement, with the syntax `c.call{value: v}(data)`, where `c` specifies the contract to which to send the amount `v` of Ether by calling the payable function whose signature as well as the passed arguments are provided in `data`.[1]

The predefined payable functions `transfer` and `send` that can be called in a more readable manner `c.transfer(v)` or `b = c.send(v)`. For sake of simplicity, we give the calculus rules for function calls along these functions.

The calculus rule for symbolically executing the `transfer` function is

$$\text{transfer (w/ callback)}\ \frac{\Gamma, c \neq \texttt{this} \Longrightarrow \{u\}\{\texttt{net(c)} := \texttt{net(c)} - \texttt{v}\}\mathrm{I}, \Delta \quad \Gamma, c \neq \texttt{this} \Longrightarrow \{u\}\{\texttt{havoc(storage)}\}(\mathrm{I} \rightarrow [\omega]\phi), \Delta}{\Gamma, c \neq \texttt{this} \Longrightarrow \{u\}[\texttt{c.transfer(v);}\ \omega]\phi, \Delta}$$

which splits the proof into two branches. The first branch establishes that invariant I of the calling contract `this` holds, before we actually execute the call, but after transferring the Ether, which is represented by updating the `net`-value of the target contract `c`. Establishing the invariant ensures that the calling contract `this` is in a consistent state when control is handed over to the called contract, and thus behaves well even in case of a callback by `c` may occur. The transfer of Ether is guaranteed by the EVM and happens before control is passed. The second branch continues symbolic execution after the calling contract regains control. Upon resuming control, we can assume that the contract's invariant holds, but we have to eliminate all knowledge about the whole storage, as nothing can be assumed about the actions of the called contract `c` before `transfer` returns. This elimination is achieved by performing a havoc operation using an update (for details how to construct such updates, see [1, Chap. 3.7.1]).

Remark 1. To prevent certain callbacks, which often lead to security holes, the current release of the Ethereum platform limits the amount of gas passed when calling `transfer`, such that the receiver is not provided with enough gas to call back. Under this assumption, the rule simplifies to

$$\text{transfer (w/o callback)}\ \frac{\Gamma, c \neq \texttt{this} \Longrightarrow \{u\}\{\texttt{net(c)} := \texttt{net(c)} - \texttt{v}\}\{\texttt{havoc(c)}\}[\omega]\phi, \Delta}{\Gamma, c \neq \texttt{this} \Longrightarrow \{u\}[\texttt{c.transfer(v);}\ \omega]\phi, \Delta}$$

As no callbacks can happen, we do not have to ensure that the invariant of `this` holds when passing control. Hence, the rule has only one premise on which symbolic execution continued directly after `transfer` returns. The only visible effect is that the `net`-value of the called contract `c` has been updated accordingly.

[1] If the external called contract is defined in the same file, or an imported file, one can also invoke a payable function via `c.f{value:v}(args)`, but that is rarely the case.

```
1 contract OneAuction {
2 /*@ invariant
3   @   address(this) != owner,
4   @   address(this) != bidder,
5   @   bid == net(bidder)+net(owner),
6   @   (\forall address a;
7   @    (a != owner && a != bidder
8   @     && a != address(this))
9   @                ==> net(a) == 0),
10  @   net(owner) <= 0,
11  @   mode==Open ==> net(owner) == 0;
12  @*/
13 // Handling mode of the auction
14 enum AuctionMode {NeverStarted, Open, Closed}
15 // Handling auction information
16 AuctionMode mode;
17 address payable owner;
18 address payable bidder;
19 uint bid;
20
21 // Modifier to check mode
22 modifier inMode(AuctionMode m) {
23  require (mode == m);
24  _;
25 }
```

```
26 modifier notBy(address c) {
27  require (msg.sender != c);
28  _;
29 }
30 ...
31 /*@ requires msg.sender != address(this);
32  @ after_success
33  @   bid > \old(bid),
34  @   net(bidder) == msg.value,
35  @   \old(bidder) != msg.sender ==>
36  @             net(\old(bidder)) == 0;
37  @*/
38 function makeBid() public payable
39  inMode(AuctionMode.Open)
40  notBy(owner) {
41    require (msg.value > bid);
42    // Transfer the old bid
43    // to the old bidder
44    uint tmp = bid;
45    bid = 0;
46    bidder.transfer(tmp);
47    // Set the new bid
48    bid = msg.value;
49    bidder = msg.sender;
50  }
51 }
```

Fig. 4. Contract OneAuction

Because the invocation of `transfer` on c did not receive enough gas for external calls, we only need to eliminate the knowledge about the state of `c`. However, we can maintain all other information about the storage, in particular about the value of the fields of `this`. Nevertheless, the problem with callbacks persists as function calls via `call` can still receive enough gas which permit callbacks.

5 Tool and Experiments

We implemented our approach as a prototype, called SolidiKeY, based on the KeY tool [1]. The user can choose between two variants of the calculus, one where `transfer` allows callbacks and one where it does not. Translating the specification into a proof-obligation is not yet automatic and has to be done manually.

We performed several experiments, to get first insights into the practical usage of our approach. We discuss one example in more detail, and give some statistical information for other examples. SolidiKeY and the examples are available at https://www.key-project.org/isola2020-smart/. Figure 4 shows a simple contract implementing the business logic for an auction that can be used to sell a single item. The contract can be in one of three states (lines 14–16). It keeps track of the `owner` selling the item, and the `bidder` who gave the highest `bid` so far.

The invariant states that the contract itself can never own itself or be the highest bidder. Further, it states that the sum of what has been payed into the contract by the highest bidder and what has been payed out to the contract's owner equals `bid`. As long as the auction is open, the equation holds because

nothing has been payed out (`net(owner)==0`). Once the auction is closed, the contract transfers the bid to the owner and sets `bid` to zero, thus re-establishing the equality. Last not least, the financial net of the contract to everyone else except for the owner and the *current* highest bidder is zero, always.

The modifiers `inMode()` and `notBy()` (lines 22–29) implement checks whether the contract is in a given mode or the caller of a method is not identical to a given address, respectively. The placeholder statement `_;` is original Solidity syntax, and refers to the body of the function to which the modifier is attached. For example, function `makeBid()` has both modifiers and their effect is the same as when inserting the two statements `require(mode==AuctionMode.Open); require(msg.sender!=owner);` at the beginning of the function body.

Function `makeBid()` is called if someone wants to place a bid. Its precondition together with the modifiers and require-check state that placing a bid is only successful if the auction is open, the bid is neither from the contract itself nor from the owner and it is higher than the previously highest bid. In that case the function shall guarantee that, after successful termination, the field `bid` has been updated to the higher bid, the caller is the new highest `bidder`, the amount of money we owe to the caller is exactly the new bid, and that we have a financial net of 0 with the previous highest bidder (unless she overbidded herself).

If `transfer()` allows callbacks, then method `makeBid()` cannot be proven correct. Neither the invariant is guaranteed to be preserved, nor the postcondition can be proven. For instance, when calling transfer to return the money to the former highest bidder, the former bidder could callback to place a higher bid. Then, once the original call resumes, that bid is overwritten. Hence, the former bidder will not get her money back. In the variant where function `transfer()` cannot call back, the function specification (incl. the invariant) can be proven. Rewriting the function `makeBid()` as shown in Fig. 5, where the call to `transfer()` is the last statement, fixes the problem.

```
function makeBid() ... {
  require (msg.value > bid);
  uint oldBid = bid;
  uint oldBidder = bidder;
  bid = msg.value;
  bidder = msg.sender;
  oldBidder.transfer(oldBid);
}
```

Fig. 5. Fixed `makeBid()`

Table 1 lists several of our verification experiments (auction-last is the fixed version of the above example). All proofs were done fully automatically. Proofs that could not be closed did point to a problem in the contract.

6 Related Work

We discuss some approaches to the analysis and verification of smart contracts.

In [10], an executable, SOS-style semantics of EVM byte code is defined in the framework K, which also generates a program verifier. The specifications and proofs are quite low level, talking about byte code, and about the artefacts of the SOS formalisation. There is ongoing work about a K semantics for Solidity.

In [4] the authors present a calculus for TinySol, a Solidity-like but much reduced language. They give a formal big-step style operational semantics that allows one to study the intricacies of Solidity semantics concerning external function calls (and hence, re-entrance) and transfer of currency. In contrast to

Table 1. Verification Results

Name	Method	Invariant verified		Postcondition verified	
		w/	w/o	w/	w/o
auction-last	transfer	✓	✓	×	✓
auction	transfer	×	✓	×	✓
auction-wd	makeBid	✓	✓	✓	✓
auction-wd	withdraw	✓	✓	×	✓
escrow	placeInEscrow	✓	✓	✓	✓
escrow	withdrawFrom	×	✓	×	✓
piggyBank	addMoney	✓	✓	✓	✓
piggyBank	break	✓	✓	×	✓

✓: Proof closed, ×: Proof open w/: transfer with callbacks, w/o: transfer without callbacks
Verification times:
(1) w/ callbacks (median/avg/min/max): 807 ms/1623 ms/272 ms/8450 ms
(2) wo callbacks (median/avg/min/max): 646 ms/1179 ms/278 ms/3593 ms
All experiments performed on macOS with an 8-core Intel i9, 2.3 GHz & warm OpenJDK 14.0.1 VM

us, they do not target verification of contracts and do not provide a program logic to explicitly specify a property about a contract to be proven.

Linters like SolCheck (https://git.io/fxXeu), Solium (https://git.io/fxXe) and others are lightweight static-analysis tools that check mainly for the occurrence of vulnerability patterns and are used to find 'smells', which may indicate bugs. They are not able to check functional correctness properties.

Oyente [12], Mayan [14], Manticore [13], and Slither [8] are symbolic execution based static analysis tools, of which some are able to check for user-specified properties. They aim to support bug finding and are in contrast to us not compositional, less expressive and bound in the symbolic execution depth. In contrast, our approach is compositional, allowing verification independent of the behaviour of external contracts.

VerX [15] is an automatic CEGAR-based verifier for Solidity contracts. They use an LTL-style specification language and their invariants must hold throughout the function execution, while ours can be invalidated in between and only must be established at certain points. For re-entrance safety they rely on checking effective external callback freedom for a given bundle. This check requires that all external contracts are known a priori.

VeriSol [11] and SOLC [9] are verification tools for Solidity both based on Boogie. Like us, these works perform deductive verification of Solidity source code, however without our focus on the payment history.

In [5] the authors extend KeY [1] to support verification of Hyperledger Fabric contracts written in Java. They focus on modelling Hyperledger's storage of objects, which is a database of serialized objects. Unlike Etherem, Hyperledger does not have a built-in payment mechanism.

Javadity [2] which we co-authored, translates Solidity contracts into Java programs that can be specified with JML and verified with out-of-the-shelf verification systems like KeY. While the approach worked in principle, the translation

overhead significantly impacted automation, and partly motivated verification that reasons directly about Solidity source code in the current work.

To our understanding, none of the above approaches provides built-in support to reason about the history of money flow. For some of them this may still be possible by manually inserting ghost annotations, which however can be prone to errors.

7 Conclusion

The smart contract domain makes up a challenging combination of different architectural principles. On the one hand, the software ecosystem is very open and highly decentralised, with different parts of the code base written by parties with potentially conflicting interests. On the other hand, the execution of the contracts remains entirely sequential, even if multiple contracts, following different goals, are involved. For the functional verification of smart contracts, it is therefore natural to use concepts which are based on principles used in both worlds: functional verification of distributed programs (e.g., [3,7]), and functional verification of sequential programs (e.g., KeY [1]). From the distributed world, we use the theme that the history of communication (in our domain, the history of payments) is a central vehicle for compositional, functional verification. At the same time, many lessons from sequential modular deductive program verification carry over to smart contracts. Two distinguished challenges are re-entrance and transfer of resources. We presented an approach that tackles both. By keeping book of the net transfer of Ether between two contracts, it is possible to elegantly specify strong data properties. By using local invariants, we keep the reasoning process modular and maintain data integrity even when calling unknown external contracts, taking into account arbitrary behaviour of the environment.

We implemented the approach in a prototype tool, SolidiKeY, and performed successful experiments to evaluate the practicality of our approach. Note that the successful proofs show more than the absence of—harmful—re-entrance attacks. They verify the invariance of contract specific relations between payment and data, which other properties, like postconditions, rely on.

More research is to be done to get a better coverage and usability, higher expressivity, and general accessibility, of formal methods for smart contracts.

Acknowledgements. We thank Gordon Pace for many fruitful discussions about Solidity verification, and for providing us with Solidity contracts on which we based our experiments. We want to highlight that we have deliberately introduced problems into these examples that were not present in Gordon's contracts.

References

1. Ahrendt, W., Beckert, B., Bubel, R., Hähnle, R., Schmitt, P.H., Ulbrich, M. (eds.): Deductive Software Verification - The KeY Book - From Theory to Practice. LNCS, vol. 10001. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-319-49812-6

2. Ahrendt, W., et al.: Verification of smart contract business logic. In: Hojjat, H., Massink, M. (eds.) FSEN 2019. LNCS, vol. 11761, pp. 228–243. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-31517-7_16
3. Ahrendt, W., Dylla, M.: A system for compositional verification of asynchronous objects. Sci. Comput. Program. **77**(12), 1289–1309 (2012)
4. Bartoletti, M., Galletta, L., Murgia, M.: A minimal core calculus for solidity contracts. In: Pérez-Solà, C., Navarro-Arribas, G., Biryukov, A., Garcia-Alfaro, J. (eds.) DPM/CBT -2019. LNCS, vol. 11737, pp. 233–243. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-31500-9_15
5. Beckert, B., Schiffl, J., Ulbrich, M.: Smart contracts: application scenarios for deductive program verification. In: Sekerinski, E., et al. (eds.) FM 2019. LNCS, vol. 12232, pp. 293–298. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-54994-7_21
6. Chittoda, J.: Mastering Blockchain Programming with Solidity. Packt (2019)
7. Din, C.C., Owe, O.: A sound and complete reasoning system for asynchronous communication with shared futures. J. Logical Algebraic Methods Program. **83**(5), 360–383 (2014)
8. Feist, J., Grieco, G., Groce, A.: Slither: A static analysis framework for smart contracts. In: Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain, WETSEB@ICSE 2019, pp. 8–15. IEEE/ACM (2019)
9. Hajdu, Á., Jovanović, D.: SOLC-VERIFY: a modular verifier for solidity smart contracts. In: Chakraborty, S., Navas, J.A. (eds.) VSTTE 2019. LNCS, vol. 12031, pp. 161–179. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-41600-3_11
10. Hildenbrandt, E., et al.: KEVM: a complete semantics of the Ethereum virtual machine. In: 2018 IEEE 31st Computer Security Foundations Symposium. IEEE (2018)
11. Lahiri, S.K., Chen, S., Wang, Y., Dillig, I.: Formal specification and verification of smart contracts for Azure blockchain. CoRR abs/1812.08829 (2018)
12. Luu, L., Chu, D., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) Proceedings of of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 254–269. ACM (2016)
13. Mossberg, M., et al.: Manticore: a user-friendly symbolic execution framework for binaries and smart contracts. In: 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, pp. 1186–1189. IEEE (2019)
14. Nikolic, I., Kolluri, A., Sergey, I., Saxena, P., Hobor, A.: Finding the greedy, prodigal, and suicidal contracts at scale. In: Proceedings of the 34th Annual Computer Security Applications Conference, ACSAC 2018, pp. 653–663. ACM (2018)
15. Permenev, A., Dimitrov, D., Tsankov, P., Drachsler-Cohen, D., Vechev, M.: VerX: safety verification of smart contracts. In: 2020 IEEE Symposium on Security and Privacy (SP), pp. 414–430. IEEE (2020)
16. Wood, G.: Ethereum: a secure decentralised generalised transaction ledger. Ethereum Proj. Yellow Pap. **151**, 1–32 (2014)